

Microsoft Graph API – Integration Reference

This document describes the full Graph API service layer at [src/server/services/graph/](#).

File Structure

```
src/server/services/graph/
├─ index.js           ← public barrel – import everything from here
├─ graphClient.js     ← OAuth2 auth flows (auth-code, refresh, app-only)
├─ graphRequest.js    ← shared fetch wrapper + pagination helper
├─ graphErrors.js     ← GraphError class, error builder
├─ tokenUtils.js      ← JWT decode, scope/role inspection, expiry check
├─ services/
│   ├─ userService.js ← User.Read, User.ReadBasic.All, User.ReadWrite
│   ├─ calendarService.js ← Calendars.* scopes
│   ├─ mailService.js ← Mail.* scopes (no send – Mail.Send not granted)
│   ├─ mailboxSettingsService.js ← MailboxSettings.ReadWrite
│   ├─ teamsChatService.js ← Chat.*, ChatMessage.* scopes
│   ├─ teamsChannelService.js ← ChannelMessage.Send, ChannelMessage.Edit
│   └─ peopleService.js ← People.Read
```

Setup – Environment Variables

Add to your `.env` (see [.env.example](#)):

```
GRAPH_TENANT_ID=162035a0-31d1-4b3c-a276-491c1dbea2f1
GRAPH_CLIENT_ID=1fb8c198-909a-412b-9b2e-20450f6e7ecc
GRAPH_CLIENT_SECRET=          ← never commit this
GRAPH_REDIRECT_URI=http://localhost:3069/auth/redirect
GRAPH_SCOPES=User.Read Mail.Read Calendars.Read Chat.Read People.Read MailboxSettings.ReadWrite offline_access
```

`GRAPH_CLIENT_SECRET` is read by `buildConfig()` in `src/server/config/env.js` as `config.graphClientSecret`.

Running the Test Script

```
# Set secret for this session (or put it in .env)
$env:GRAPH_CLIENT_SECRET = "your-secret"

# All tests – opens browser once, runs everything
node scripts/test-graph-api.js

# Single test
node scripts/test-graph-api.js me
node scripts/test-graph-api.js messages
node scripts/test-graph-api.js inbox
node scripts/test-graph-api.js unread
node scripts/test-graph-api.js message-search
node scripts/test-graph-api.js mailbox
node scripts/test-graph-api.js calendar
node scripts/test-graph-api.js calendar-view
node scripts/test-graph-api.js calendar-today
node scripts/test-graph-api.js chats
node scripts/test-graph-api.js chat-messages
node scripts/test-graph-api.js people
node scripts/test-graph-api.js users
node scripts/test-graph-api.js search-users
node scripts/test-graph-api.js token
```

Auth Flows

1. Interactive (delegated) – most common

```
import { acquireTokenInteractive } from "../services/graph/graphClient.js";

const tokenSet = await acquireTokenInteractive({
  tenantId:    config.graphTenantId,
  clientId:    config.graphClientId,
  clientSecret: config.graphClientSecret,
  redirectUri: config.graphRedirectUri,
  scopes:      config.graphScopes,
});
// tokenSet.accessToken ← use for all Graph calls
// tokenSet.refreshToken ← save securely for silent renewal
```

Opens browser → user signs in → redirected back → code exchanged for tokens.

2. Refresh (silent renewal)

```
import { refreshAccessToken } from "../services/graph/graphClient.js";
import { isTokenExpired }    from "../services/graph/tokenUtils.js";

if (isTokenExpired(savedAccessToken)) {
  const newTokenSet = await refreshAccessToken({
    ...authCfg,
    refreshToken: savedRefreshToken,
  });
}
```

Requires `offline_access` scope. Runs silently – no browser needed.

3. App-only (client credentials)

```
import { acquireAppToken } from "../services/graph/graphClient.js";

const tokenSet = await acquireAppToken({
  tenantId:    config.graphTenantId,
  clientId:    config.graphClientId,
  clientSecret: config.graphClientSecret,
});
```

No user – use `/users/{id}` endpoints, not `/me`. Requires application permissions (not delegated) to be configured in the Azure app registration.

4. Manual auth-code flow (for Express route integration)

```
import { buildAuthorizationUrl, exchangeCodeForToken } from "../services/graph/graphClient.js";

// Step 1 – redirect user
const { url, state } = buildAuthorizationUrl({ ...authCfg });
req.session.oauthState = state;
res.redirect(url);

// Step 2 – handle redirect callback
router.get("/auth/redirect", async (req, res) => {
  const { code, state } = req.query;
  if (state !== req.session.oauthState) return res.status(403).send("State mismatch");
  const tokenSet = await exchangeCodeForToken({ ...authCfg, code });
  req.session.graphToken = tokenSet;
  res.redirect("/");
});
```

TokenSet shape

```
{
  accessToken: string, // Bearer token for Graph API calls (~1 hour)
  refreshToken: string, // For silent renewal (persistent until revoked)
```

```

idToken:    string,    // JWT with user identity claims
expiresIn:  number,    // Seconds until accessToken expires
expiresAt:  number,    // Date.now() value when it expires
scope:      string,    // Space-separated scopes that were granted
}

```

Token Utilities

```

import { decodeTokenClaims, getTokenScopes, hasScope, isTokenExpired } from "../services/graph/tokenUtils.js";

decodeTokenClaims(token)    // → JWT payload object (no signature verification)
getTokenScopes(token)       // → ["User.Read", "Mail.Read", ...] (delegated tokens)
getTokenRoles(token)        // → ["Mail.Read", ...] (app tokens)
hasScope(token, "Mail.Read") // → true / false
isTokenExpired(token)       // → true if within 60s of expiry
isDelegatedToken(token)     // → true if token has `scp` claim (user token)

```

Service Function Reference

userService

Function	Scope	Description
<code>getMe(token, { select })</code>	User.Read	Signed-in user profile
<code>getMyPhotoMetadata(token)</code>	User.Read	Photo dimensions + mime type
<code>getMyPhotoValue(token)</code>	User.Read	Raw photo binary (returns Response)
<code>listUsers(token, { top, select, filter, orderby })</code>	User.ReadBasic.All	All users in tenant
<code>searchUsers(token, text, { select, top })</code>	User.ReadBasic.All	Full-text display name search
<code>getUser(token, userIdOrUpn, { select })</code>	User.ReadBasic.All	Single user by ID or UPN
<code>updateUser(token, userIdOrUpn, patch)</code>	User.ReadWrite	Partial update (safe fields only)

```

import * as userService from "../services/graph/services/userService.js";

const me = await userService.getMe(token, {
  select: "id,displayName,mail,jobTitle,officeLocation",
});

const users = await userService.listUsers(token, { top: 50 });
const user = await userService.getUser(token, "person@company.com");
await userService.updateUser(token, "person@company.com", { jobTitle: "Lead Developer" });

```

calendarService

Function	Scope	Description
<code>listMyCalendars(token, { select })</code>	Calendars.ReadBasic	All user calendars
<code>listMyCalendarEvents(token, { top, select, filter, orderby })</code>	Calendars.Read	Events from primary calendar
<code>getEvent(token, eventId, { select })</code>	Calendars.Read	Single event
<code>getMyCalendarView(token, start, end, opts)</code>	Calendars.ReadBasic	Events in date range
<code>listSharedCalendarEvents(token, mailbox, opts)</code>	Calendars.Read.Shared	Events from shared calendar
<code>getSharedCalendarView(token, mailbox, start, end, opts)</code>	Calendars.Read.Shared	Shared calendar date range
<code>createEvent(token, eventInput)</code>	Calendars.ReadWrite	Create event
<code>updateEvent(token, eventId, patch)</code>	Calendars.ReadWrite	Partial update
<code>deleteEvent(token, eventId)</code>	Calendars.ReadWrite	Delete event

```
import * as calendarService from "../services/graph/services/calendarService.js";

// Today's events
const today = new Date();
const tomorrow = new Date(today);
tomorrow.setDate(today.getDate() + 1);
const events = await calendarService.getMyCalendarView(
  token,
  today.toISOString(),
  tomorrow.toISOString(),
  { select: "id,subject,start,end,location", timezone: "Eastern Standard Time" }
);

// Create an event
await calendarService.createEvent(token, {
  subject: "Graph API Test Meeting",
  body: { contentType: "HTML", content: "Test event created via Graph API." },
  start: { dateTime: "2026-06-01T14:00:00", timeZone: "Eastern Standard Time" },
  end: { dateTime: "2026-06-01T14:30:00", timeZone: "Eastern Standard Time" },
  isOnlineMeeting: true,
  onlineMeetingProvider: "teamsForBusiness",
});
```

mailService

Function	Scope	Description
listMyMessagesBasic(token, opts)	Mail.ReadBasic	Messages, metadata only
listInboxMessages(token, opts)	Mail.ReadBasic	Inbox
getMessage(token, messageId, opts)	Mail.Read	Full message with body
searchMyMessages(token, text, opts)	Mail.Read	Full-text mail search
listUnreadMessages(token, opts)	Mail.ReadBasic	Unread only
listAllMessages(token, opts)	Mail.Read	All pages, merged
listSharedMailboxMessages(token, mailbox, opts)	Mail.Read.Shared	Shared mailbox
listSharedInboxMessages(token, mailbox, opts)	Mail.Read.Shared	Shared mailbox inbox
createDraftMessage(token, draftInput)	Mail.ReadWrite	Create draft
createSharedMailboxDraft(token, mailbox, draftInput)	Mail.ReadWrite.Shared	Draft in shared mailbox
updateDraftMessage(token, messageId, patch)	Mail.ReadWrite	Update draft
setMessageReadState(token, messageId, isRead)	Mail.ReadWrite	Mark read/unread
moveMessage(token, messageId, destinationId)	Mail.ReadWrite	Move to folder
deleteMessage(token, messageId)	Mail.ReadWrite	Delete message

No `sendMessage()` function. `Mail.Send` is not in the granted scope list. Add it to the Azure app registration first, then implement it here.

```
import * as mailService from "../services/graph/services/mailService.js";

// Recent inbox with full body
const inbox = await mailService.listInboxMessages(token, {
  top: 25,
  select: "id,subject,from,receivedDateTime,isRead,body",
});

// Search
const results = await mailService.searchMyMessages(token, "expense report", { top: 10 });

// Mark read
await mailService.setMessageReadState(token, messageId, true);
```

```
// Move to archive
await mailService.moveMessage(token, messageId, "Archive");
```

mailboxSettingsService

Function	Scope	Description
getMyMailboxSettings(token)	MailboxSettings.ReadWrite	All settings
updateMyMailboxSettings(token, patch)	MailboxSettings.ReadWrite	Partial update

```
import * as mailboxSettingsService from "../services/graph/services/mailboxSettingsService.js";

// Get settings
const settings = await mailboxSettingsService.getMyMailboxSettings(token);

// Enable scheduled auto-reply
await mailboxSettingsService.updateMyMailboxSettings(token, {
  automaticRepliesSetting: {
    status: "scheduled",
    scheduledStartDateTime: { dateTime: "2026-06-01T18:00:00", timeZone: "Eastern Standard Time" },
    scheduledEndDateTime: { dateTime: "2026-06-05T08:00:00", timeZone: "Eastern Standard Time" },
    internalReplyMessage: "I'm out of office until June 5.",
    externalReplyMessage: "I'm out of office until June 5.",
  },
});

// Set timezone
await mailboxSettingsService.updateMyMailboxSettings(token, { timeZone: "Eastern Standard Time" });
```

teamsChatService

Function	Scope	Description
listMyChats(token, { top, select })	Chat.ReadBasic	User's chats
getChat(token, chatId, { select })	Chat.Read	Single chat
listChatMembers(token, chatId)	Chat.ReadBasic	Chat members
createOneOnOneChat(token, userIdA, userIdB)	Chat.Create	1:1 chat
createGroupChat(token, topic, userIds)	Chat.Create	Group chat
listChatMessages(token, chatId, { top })	ChatMessage.Read	Messages in chat
listAllChatMessages(token, chatId)	ChatMessage.Read	All pages
getChatMessage(token, chatId, messageId)	ChatMessage.Read	Single message
sendChatMessage(token, chatId, content, contentType)	ChatMessage.Send	Send message

```
import * as teamsChatService from "../services/graph/services/teamsChatService.js";

// Find a chat and read messages
const chats = await teamsChatService.listMyChats(token, { top: 10 });
const chatId = chats[0].id;
const messages = await teamsChatService.listChatMessages(token, chatId, { top: 20 });

// Send a message
await teamsChatService.sendChatMessage(token, chatId, "<b>Hello</b> from the portal app!", "html");

// Create a 1:1 chat (need user object IDs – get from userService)
const newChat = await teamsChatService.createOneOnOneChat(token, userId1, userId2);
```

teamsChannelService

Requires [teamId](#) and [channelId](#) – get these from the Teams admin portal or the Graph Teams APIs (requires additional scopes like [Team.ReadBasic.All](#)).

Function	Scope	Description
<code>sendChannelMessage(token, teamId, channelId, content, contentType, extra)</code>	ChannelMessage.Send	Post to channel
<code>sendChannelReply(token, teamId, channelId, messageId, content, contentType)</code>	ChannelMessage.Send	Reply to thread
<code>editChannelMessage(token, teamId, channelId, messageId, content, contentType)</code>	ChannelMessage.Edit	Edit own message
<code>editChannelReply(token, teamId, channelId, messageId, replyId, content, contentType)</code>	ChannelMessage.Edit	Edit own reply

```
import * as teamsChannelService from "../services/graph/services/teamsChannelService.js";

await teamsChannelService.sendChannelMessage(
  token,
  "TEAM-ID-HERE",
  "CHANNEL-ID-HERE",
  "<b>Automated report</b> from the portal app.",
  "html"
);
```

peopleService

Function	Scope	Description
<code>listRelevantPeople(token, { top, select, filter })</code>	People.Read	Ranked relevant people
<code>searchPeople(token, text, { top, select })</code>	People.Read	Search by name/email

```
import * as peopleService from "../services/graph/services/peopleService.js";

const people = await peopleService.listRelevantPeople(token, { top: 10 });
const results = await peopleService.searchPeople(token, "Mataan");
```

Error Handling

All service functions throw `GraphError` on non-2xx responses.

```
import { GraphError } from "../services/graph/graphErrors.js";

try {
  const me = await userService.getMe(token);
} catch (err) {
  if (err instanceof GraphError) {
    console.log(err.status);      // 401, 403, 404, 429, 500, ...
    console.log(err.graphCode);   // e.g. "InvalidAuthenticationToken"
    console.log(err.graphMessage); // Human-readable Graph error message
    console.log(err.requestId);   // MS request ID for support tickets

    if (err.isUnauthorized()) // 401 – token expired or wrong audience
    if (err.isForbidden())    // 403 – missing consent or permission
    if (err.isNotFound())     // 404 – wrong ID or no access
    if (err.isThrottled())    // 429 – slow down (graphRequest auto-retries)
    if (err.isServerError())  // 5xx (graphRequest auto-retries 502/503/504)
  }
}
```

The shared `graphRequest` automatically retries `429`, `502`, `503`, `504` up to 3 times with exponential backoff. It never retries `400`, `401`, `403`, or `404`.

Pagination

By default, Graph returns a single page. Two options:

```
// Option A: single page (use $top to control size)
const msgs = await mailService.listInboxMessages(token, { top: 25 });

// Option B: all pages merged (use carefully on large datasets)
const allMsgs = await mailService.listAllMessages(token, { folder: "inbox" });
```

You can also call the low-level helper directly:

```
import { graphGetAllPages } from "../services/graph/graphRequest.js";

const allUsers = await graphGetAllPages({
  method: "GET",
  path: "/users",
  token,
  query: { $select: "id,displayName,mail", $top: 100 },
});
```

Integrating into Express Routes

When you're ready to wire this into the app:

```
// src/server/routes/graph.js
import { createAuthClient } from "../services/graph/graphClient.js";
import * as mailService from "../services/graph/services/mailService.js";

export function createGraphRoutes(config) {
  const auth = createAuthClient({
    tenantId: config.graphTenantId,
    clientId: config.graphClientId,
    clientSecret: config.graphClientSecret,
    redirectUri: config.graphRedirectUri,
    scopes: config.graphScopes,
  });

  const router = express.Router();

  router.get("/auth/login", (req, res) => {
    const { url, state } = auth.buildAuthorizationUrl();
    req.session.oauthState = state;
    res.redirect(url);
  });

  router.get("/auth/redirect", async (req, res, next) => {
    try {
      const { code, state } = req.query;
      if (state !== req.session.oauthState) return res.status(403).send("State mismatch");
      const tokenSet = await auth.exchangeCodeForToken(code);
      req.session.graphToken = tokenSet;
      res.redirect("/");
    } catch (err) { next(err); }
  });

  router.get("/api/graph/messages", async (req, res, next) => {
    try {
      const token = req.session.graphToken?.accessToken;
      if (!token) return res.status(401).json({ error: "Not authenticated" });
      const messages = await mailService.listInboxMessages(token, { top: 20 });
      res.json(messages);
    } catch (err) { next(err); }
  });

  return router;
}
```

Keep `GRAPH_CLIENT_SECRET` server-side only – never expose it to the frontend.

Excluded Scopes

These scopes are in the Azure app registration but are **not implemented**:

```
SecurityCopilotWorkspaces.Read.All  
SecurityCopilotWorkspaces.ReadWrite.All  
CopilotPackages.Read.All
```

Add service functions for them only if explicitly needed.

Also excluded (not in the granted scope list):

```
Mail.Send ← mailService has no sendMessage() until this is granted
```